西南大学

智能传感与检测系统

课程报告

题　目:基于 ROS2 的轮式机器人 SLAM 建图与自主导航

学生姓名：余佳欣

专　　业：智能制造

学　　号：222024320250052

班　　级：2024 级科创班

指导教师：冀杰

成　　绩：

工程技术学院

2026 年 6 月

目 录

一、作业概述 4
二、技术路线 4
步骤 1：URDF 机器人建模 4
步骤 2：Gazebo 仿真场景搭建 4
步骤 3：SLAM 实时建图 4
步骤 4：Nav2 导航规划 5
步骤 5：路径规划与跟踪 5
步骤 6：录屏与报告整理 5
三、核心代码说明 5
3.1 机器人模型（robot.xacro） 5
3.2 仿真场景（scene.world） 6
3.3 Launch 文件 7
3.4 SLAM 参数配置（slam_params.yaml） 7
3.5 Nav2 导航参数配置（nav2_params.yaml） 8
四、调试过程 9
4.1 URDF 模型调试 9
4.2 SLAM 建图参数调优 9
4.3 Nav2 导航参数调优 9
五、遇到的问题及解决方法 9
激光雷达采样密度不足 10
建图过程中地图更新滞后 10
窄通道路径规划失败 10
AMCL 定位漂移 10

机器人卡在障碍物附近 10
六、作业总结 10
附录：**核心参数**汇总 12

一、作业概述

本实验基于 ROS2 Humble 系统，在 Gazebo 仿真环境中完成了一款差速驱动轮式机器人的建模、环境地图构建与自主导航任务。作业的核心目标是掌握机器人 URDF 建模方法、激光雷达 SLAM 建图技术以及 Nav2 导航框架的完整使用流程，最终实现机器人在包含障碍物的场地中从入口自主导航至出口。

实验场地为一个 8 米 × 5 米的封闭矩形区域，四周由红色围墙限定，左侧留有入口（y 方向 1.9 m ~ 3.1 m），右侧留有出口（位置对称）。场地内布置了 5 个障碍物：2 个 0.6 m 立方体分别位于场地左半区上下两侧，2 个 0.8 m × 0.4 m × 0.3 m 的长方体位于中部，以及 1 个直径 0.5 m 的圆柱体靠近右侧出口区域。障碍物之间的通道最窄处约为 1.0 m，机器人需具备一定的路径规划能力。

机器人本体采用差速驱动结构，包含两个主动驱动轮和一个后方万向从动轮。车身呈墨绿色金属质感，尺寸为 0.46 m × 0.34 m × 0.22 m，顶部前方安装有单线激光雷达传感器，具备 480 个采样点、4.0 m 有效量程和 360 度水平扫描能力，为 SLAM 建图和导航避障提供感知数据。

二、技术路线

本实验的完整技术流程分为以下六个步骤：

步骤 1：URDF 机器人建模

使用 xacro 宏语言编写机器人 URDF 模型，定义车身、驱动轮、万向轮和激光雷达的几何尺寸、碰撞属性、惯性参数和视觉材质。通过 Gazebo 插件为机器人添加差速驱动控制器和激光雷达传感器。

步骤 2：Gazebo 仿真场景搭建

使用 SDF 格式构建 8×5 m 的仿真场地，包含围墙和 5 个障碍物的精确位置与物理属性。设置地面摩擦系数、重力参数和物理引擎更新频率。

步骤 3：SLAM 实时建图

启动 slam_toolbox 的异步模式，订阅激光雷达 /scan 话题，在机器人遍历环境的过程中增量式构建二维占据栅格地图。设定地图分辨率为 0.04 m，建图更新间隔 0.8 秒。

步骤 4：Nav2 导航规划

加载已构建的地图，启动 AMCL 进行全局定位。基于 NavFn 规划器（A* 算法）计算全局路径，使用 DWB 局部规划器进行实时路径跟踪和动态避障。

步骤 5：路径规划与跟踪

在 RViz 中设定初始位姿和导航目标点，观察机器人从入口自动规划路径，依次绕过所有障碍物到达出口。

步骤 6：录屏与报告整理

录制完整的操作演示视频，整理提交所有核心代码文件，撰写课程报告总结实验过程与技术要点。

三、核心代码说明

3.1 机器人模型（robot.xacro）

机器人模型定义了车身、驱动轮、万向轮和激光雷达的结构。以下为车身与差速驱动插件的核心代码片段：

```
<!-- 车身本体：绿色金属，0.46×0.34×0.22m，质量 1.6kg -->
<link name="base_link">
  <inertial>
    <mass value="1.6"/>
    <origin xyz="0 0 0.11"/>
    <inertia ixx="0.02373" ixy="0.0" ixz="0.0"
             iyy="0.03213" iyz="0.0" izz="0.03477"/>
  </inertial>
  <visual>
    <origin xyz="0 0 0.11"/>
    <geometry><box size="0.46 0.34 0.22"/></geometry>
    <material name="green_metal"/>
  </visual>
</link>
```

驱动轮连接在车身两侧前方，轮距 0.32 m。激光雷达安装在车顶前方 6 cm 处，采样 480 个点、量程 4.0 m：

```
<!-- 左驱动轮：半径 0.08m，宽度 0.04m，Y 轴旋转 -->
<joint name="left_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="left_wheel_link"/>
  <origin xyz="0.14 0.16 0.07" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <limit effort="12" velocity="12"/>
```

```
</joint>

<!-- 激光雷达：480 采样点，360° 扫描，4.0m 量程 -->
<gazebo reference="laser_link">
  <sensor type="ray" name="lidar_sensor">
    <update_rate>30</update_rate>
    <ray>
      <scan><horizontal>
        <samples>480</samples>
        <min_angle>${-pi}</min_angle>
        <max_angle>${pi}</max_angle>
      </horizontal></scan>
      <range><min>0.1</min><max>4.0</max></range>
    </ray>
    <plugin name="gazebo_ros_lidar" filename="libgazebo_ros_ray_sensor.so">
      <argument>~/out:=/scan</argument>
    </plugin>
  </sensor>
</gazebo>
```

差速驱动控制器配置车轮分离距离和 PID 参数：

```
<plugin name="gazebo_ros_diff_drive" filename="libgazebo_ros_diff_drive.so">
  <left_joint>left_wheel_joint</left_joint>
  <right_joint>right_wheel_joint</right_joint>
  <wheel_separation>0.32</wheel_separation>
  <wheel_diameter>0.16</wheel_diameter>
  <max_wheel_torque>25</max_wheel_torque>
  <p_gain>85.0</p_gain><i_gain>8.0</i_gain><d_gain>1.2</d_gain>
  <command_topic>/cmd_vel</command_topic>
  <publish_odom>true</publish_odom>
</plugin>
```

3.2 仿真场景（scene.world）

仿真场景定义了 8×5 m 场地、围墙和 5 个障碍物。以下为障碍物布局的核心代码：

```
<!-- 障碍物 1：立方体 0.6m @ (2.5, 1.5) -->
<model name="obstacle_1">
  <static>true</static>
  <pose>2.5 1.5 0.3 0 0 0</pose>
  <link name="link">
    <collision><geometry><box><size>0.6 0.6 0.6</size></box></geometry></collision>
    <visual><geometry><box><size>0.6 0.6 0.6</size></box></geometry></visual>
  </link>
</model>

<!-- 障碍物 2：立方体 @ (2.5, 3.5) -->
<model name="obstacle_2">
  <pose>2.5 3.5 0.3 0 0 0</pose>
  ...（尺寸同上）
</model>
```

```
<!-- 障碍物 3-4：长方体 0.8×0.4×0.3m @ (4.5, 2.0) 和 (4.5, 3.0) -->
<!-- 障碍物 5：圆柱体 d0.5×0.3m @ (6.0, 2.5) -->
```

3.3 Launch 文件

启动文件是整个实验流程的入口，负责启动 Gazebo、生成机器人、加载 SLAM 或 Nav2 导航。以下为建图启动文件 mapping.launch.py 和导航启动文件 navigation.launch.py 的核心内容：

mapping.launch.py（建图模式）：

```
def generate_launch_description():
    pkg_gazebo_scene = get_package_share_directory('gazebo_scene')
    return LaunchDescription([
        # 1. 启动 Gazebo 场景 + 机器人
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(pkg_gazebo_scene, 'launch', 'gazebo_scene.launch.py'))),
        # 2. 启动 SLAM 建图
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(get_package_share_directory('slam_bringup'), 'launch',
'slam.launch.py'))),
        # 3. 启动 RViz 可视化
        Node(package='rviz2', executable='rviz2',
             arguments=['-d', os.path.join(pkg_gazebo_scene, 'config', 'mapping.rviz')]),
    ])
```

navigation.launch.py（导航模式）：

```
def generate_launch_description():
    pkg_gazebo_scene = get_package_share_directory('gazebo_scene')
    pkg_nav_bringup = get_package_share_directory('nav_bringup')
    map_yaml = LaunchConfiguration("map", default=os.path.join(pkg_nav_bringup, "maps",
"map.yaml"))
    return LaunchDescription([
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(os.path.join(pkg_gazebo_scene, 'launch',
'gazebo_scene.launch.py'))),
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(os.path.join(pkg_nav_bringup, 'launch',
'nav_bringup.launch.py')),
            launch_arguments={"map": map_yaml}.items()),
        Node(package='rviz2', executable='rviz2',
             arguments=['-d', os.path.join(pkg_gazebo_scene, 'config', 'navigation.rviz')]),
    ])
```

3.4 SLAM 参数配置（slam_params.yaml）

SLAM 建图采用 slam_toolbox 的 CeresSolver 求解器，核心参数如下：

```
slam_toolbox:
  ros__parameters:
    solver_plugin: solver_plugins::CeresSolver
    resolution: 0.04                    # 地图分辨率
    map_update_interval: 0.8            # 更新间隔（秒）
    max_laser_range: 3.5                # 激光最大有效量程
    minimum_travel_distance: 0.08       # 最小移动触发距离
    minimum_travel_heading: 0.08        # 最小转向触发角度
    scan_buffer_size: 15                # 扫描历史缓冲区大小
    do_loop_closing: true               # 启用闭环检测
    throttle_scans: 1                   # 不跳过扫描帧
```

与默认参数相比，分辨率从 0.05 m 提高到 0.04 m 使地图细节更丰富；更新间隔从 3.0 s 缩短至 0.8 s 提升了建图实时性；最小移动距离和转向角度从 0.5 降至 0.08，确保小幅度运动也能触发更新。

3.5 Nav2 导航参数配置（nav2_params.yaml）

Nav2 导航堆栈配置了 AMCL 定位、全局/局部代价地图和 DWB 规划器。关键参数如下：

```
amcl:
  ros__parameters:
    max_particles: 2500                 # 粒子数上限
    min_particles: 600                  # 粒子数下限
    laser_max_range: 4.0                # 激光最大量程
    update_min_d: 0.2                   # 最小位移更新
    update_min_a: 0.15                  # 最小角度更新

local_costmap:
  local_costmap:
    ros__parameters:
      robot_radius: 0.23                # 机器人半径
      resolution: 0.04                  # 代价地图分辨率
      inflation_layer:
        inflation_radius: 0.25          # 膨胀半径
        cost_scaling_factor: 6.0        # 代价缩放因子

controller_server:
  ros__parameters:
    FollowPath:
      max_vel_x: 0.4                    # 最大线速度
      max_vel_theta: 0.8                # 最大角速度
      acc_lim_x: 2.0                    # 线加速度
      acc_lim_theta: 2.5                # 角加速度
```

膨胀半径从默认的 0.55 m 减小至 0.25 m，同时将 cost_scaling_factor 从 5.0 提高至 6.0，使得障碍物附近的代价衰减更快，1.0 m 宽的窄通道不再被膨胀层完全覆盖，机器人可以顺利通过。

四、调试过程

4.1 URDF 模型调试

在模型构建过程中，首先遇到的是车轮关节轴方向问题。初始配置时车轮旋转轴未正确对齐，导致 Gazebo 中车轮旋转方向与预期不符。通过调整 <axis> 标签的 xyz 参数，明确指定左轮和右轮的旋转轴为 Y 轴方向（xyz="0 1 0"），同时调整 visual 标签中的 rpy 旋转参数（绕 X 轴旋转 -π/2），使圆柱体从 Z 轴方向旋转到 Y 轴方向，正确呈现车轮的滚动效果。

另一个关键调试是万向轮的惯性参数修正。初始阶段使用了圆柱体惯性公式计算球形从动轮的惯量，导致物理仿真中万向轮运动异常。通过编写 inertia_sphere 宏，采用球体惯量公式（$I = 2/5 \times m \times r^2$）重新计算，解决了该问题。

4.2 SLAM 建图参数调优

在建图阶段，初始参数设置导致地图更新缓慢且质量不佳。通过以下调整显著提升了建图效果：将最小移动距离从 0.5 m 降至 0.08 m，最小转向角度阈值从 0.5 rad 降至 0.08 rad，使得机器人在小幅度移动和转弯时也能触发地图更新；将地图更新间隔从 3.0 秒缩短至 0.8 秒，提高了地图构建的实时性；将分辨率从 0.05 m 提高到 0.04 m，使地图细节更丰富。

4.3 Nav2 导航参数调优

导航调试阶段主要关注机器人在障碍物之间的通行能力。初始的膨胀半径为 0.55 m，导致障碍物之间的 1.0 m 窄通道被判定为不可通行区域。通过将膨胀半径减小至 0.25 m，同时将 cost_scaling_factor 从 5.0 提升至 6.0，有效平衡了安全距离和通道通行性。此外，将机器人半径从 0.25 m 调整为 0.23 m，进一步增加了机器人在狭窄区域中的容差。将最大平移速度从 0.5 m/s 降至 0.4 m/s，提高了导航过程中的稳定性和安全性。

五、遇到的问题及解决方法

在实验过程中，遇到并解决了以下几类典型问题：

激光雷达采样密度不足

解决方法：初始配置为 360 个采样点，角度分辨率 1 度，导致建图时障碍物边缘锯齿明显、狭小通道检测不准确。解决方案是将采样点增加到 480 个（角度分辨率约 0.75 度），同时适当提高激光雷达的测量噪声标准差至 0.002 m，在增加细节的同时保证数据的鲁棒性。

建图过程中地图更新滞后

解决方法：初始的最小移动距离和最小转向角度阈值过大（均为 0.5），导致机器人在缓慢移动和转向时地图无法及时更新，出现建图空白区域。通过将两者减小至 0.08，配合缩短的建图更新间隔 （0.8 秒），实现了平滑连续的地图构建。

窄通道路径规划失败

解决方法：障碍物 3 和障碍物 4 之间约 1.0 m 的通道在代价地图中被膨胀层覆盖，全局规划器无法找到穿越路径。通过调整膨胀半径（0.55 → 0.25 m）和成本缩放因子（5.0 → 6.0），在保证安全性的前提下成功开放了狭窄通道。

AMCL 定位漂移

解决方法：在导航启动阶段，AMCL 的初始定位如果设置不够精确，会导致机器人在地图上的位姿与实际位置偏差较大。通过增加粒子数上限（2000 → 2500）和下限（500 → 600），同时调整更新阈值（update_min_d: 0.20 m, update_min_a: 0.15 rad），使定位收敛更快、更稳定。

机器人卡在障碍物附近

解决方法：在局部路径跟踪过程中，DWB 规划器偶尔会选择接近障碍物的轨迹，导致机器人碰撞后卡住。通过将 BaseObstacle 评价函数的权重从 0.05 提升至 0.06，同时减小最大线速度（0.5 → 0.4 m/s），使机器人在接近障碍物时有更充裕的反应时间。

六、作业总结

通过本次实验，完整掌握了基于 ROS2 的机器人仿真与自主导航技术流程。从 URDF 建模到 Gazebo 仿真场景搭建，从 SLAM 实时建图到 Nav2 路径规划与跟踪，每一个环节都涉及大量参数调试和问题排查，加深了对机器人感知、定位和导航原理的理解。

在技术层面，实现了以下几点核心成果：第一，独立完成了一套完整的轮式机器人URDF 模型，包含精确的物理参数和传感器配置；第二，通过 SLAM 技术构建了包含 5 个障碍物环境的二维栅格地图；第三，成功配置 Nav2 导航堆栈的参数，实现了机器人在复杂环境中从入口到出口的自主路径规划和避障导航。

实验中遇到的各类问题及解决过程，尤其是参数调优方面的经验，对于理解机器人定位与导航算法的工程实践具有重要价值。这些经验不仅是本实验的成果，也为后续更复杂的机器人系统开发打下了良好的基础。

附录：核心参数汇总

表 1：机器人模型关键参数

参数名称	参数值	说明
车身尺寸	0.46 × 0.34 × 0.22 m	墨绿色金属材质
车身质量	1.6 kg	含传感器与车轮
驱动轮半径	0.08 m	左右各一，黑色橡胶
轮距	0.32 m	差速驱动控制
万向轮半径	0.045 m	球体，后方安装
激光雷达采样点	480	360° 扫描
激光雷达量程	4.0 m	有效检测距离

表 2：SLAM 建图关键参数

参数名称	参数值	说明
地图分辨率	0.04 m	高于默认 0.05 m
建图更新间隔	0.8 s	增量更新频率
最小移动距离	0.08 m	触发地图更新
最小转向角度	0.08 rad	触发地图更新
闭环检测	启用	减少累积误差

表 3：Nav2 导航关键参数

参数名称	参数值	说明
机器人半径	0.23 m	代价地图中使用
最大线速度	0.4 m/s	安全行驶速度
最大角速度	0.8 rad/s	转弯速度限制
膨胀半径	0.25 m	全局/局部一致
成本缩放因子	6.0	代价衰减速率
AMCL 粒子数	600~2500	自适应粒子滤波